

Pre - Understanding

After deciding to recreate DeepSea, I first familiarized myself with the conceptual underpinnings of the model. I first began by understanding the model's input. DeepSEA processes genomic data in windows of 1,000 base-pair (bp) sequences. The 200 bp center serves as the core focus area where the model looks to identify specific sequence motifs tied to transcription factors or regions of "open" chromatin. The surrounding DNA context (400 bp on each side) provides broader genomic context for the center.

Because deep learning models cannot process raw text strings of nucleotides (A, C, G, T), the sequence must be numericalized using one-hot encoding. This process maps each nucleotide to a four-dimensional vector where a single position is activated:

$$A = \langle 1, 0, 0, 0 \rangle; \quad C = \langle 0, 1, 0, 0 \rangle; \quad G = \langle 0, 0, 1, 0 \rangle; \quad T = \langle 0, 0, 0, 1 \rangle$$

In turn, the "X" input for the neural network is a matrix of the shape $\langle \text{num samples}, 4, 1000 \rangle$, and the "Y" input is $\langle \text{num samples}, \text{num_factors} \rangle$.

Once the one-hot encoded matrix passes into the network, it is analyzed by three convolutional layers, non-linear activation functions, and max pooling layers. Based on my understanding, a convolutional layer applies a series of learned filters that slide across the sequence matrix. Biologically, these filters function like digital Position Weight Matrices, scanning the DNA to detect specific sequence motifs. The layer calculates the dot product between its weights and the sequence window it covers; a high dot product signifies a strong sequence match to a learned motif. Between layers, Rectified Linear Unit (ReLU) activations introduce non-linearity, allowing the model to learn complex combinations of motifs. To optimize the network weights, the model uses multi-label binary cross-entropy as its loss function, essentially it tries to optimize the fit to the ReLU informed-graph. Following the convolutions, Max Pooling layers reduce the spatial dimension by retaining only the maximum activation values within a specified window. Pooling inspires both efficiency, as less data is processed at each layer, and also translation invariance, as the positions of certain motifs change, allowing the neural network to identify the motif regardless of its exact position. Lastly, we apply the sigmoid activation function at the very final layer that picks a score from 0 to 1 regarding each feature for every 1000 bp sample, a good choice for such multi-label classification.

On the biological side, the model predicts chromatin features, specifically, the binding of regulatory proteins and local modifications to chromatin structures (the packaging of DNA around histone proteins). For this implementation, the focus is narrowed to five foundational epigenetic marks: CTCF Binding, H3K4me3, H3K27ac, H3K4me1, and DNase accessibility. CTCF is a protein that binds onto specific sequences, helps direct chromatin loop extrusion, loops together distant regions of DNA, and separates certain regulatory regions; its sequence specificity makes it a great fit for such a model. H3K4me3 indicates tri-methylation of Lysine 4 on Histone H3. Positioned tightly around Transcription Start Sites, this modification acts as a docking site for "reader" proteins that recruit RNA Polymerase II and the core transcription

machinery to initiate gene transcription. Similarly, H3K27ac denotes the acetylation of Lysine 27 on Histone H3. Chemically, lysine residues carry a positive charge that tightly binds the negatively charged phosphate backbone of DNA. Acetylation neutralizes this positive charge, loosening the DNA-histone grip and making the region physically accessible. H3K27ac marks active enhancers and promoters. If you see H3K27ac without H3K4me3 nearby, that's almost certainly an enhancer rather than a promoter. H3K4me1 represents mono-methylation of Lysine 4 on Histone H3. It acts as a signature for enhancer regions that are "primed" and ready for activation but waiting for downstream activation signals (like the addition of H3K27ac). Lastly, DNase is a measurement of physical chromatin accessibility. Regions of DNA that are not tightly wrapped around histones are highly vulnerable to cleavage by the DNase I enzyme, highlighting "open" regulatory hubs.

- Active enhancers: H3K4me1 + H3K27ac + DNase accessible (all three together)
- Active promoters: H3K4me3 + H3K27ac + DNase accessible
- Poised enhancers: H3K4me1, *no* H3K27ac, often *still* DNase accessible (just less so)
- Repressed regions: H3K27me3, *no* DNase accessibility

Data Processing

For this study, I used a the chr21 FASTA file, a text-based file containing the raw genetic code for human Chromosome 21, formatted in the standard bioinformatics, complimented by ENCODE peak files for GM12878, which are precomputed datasets that map our five factors across the genome of this widely used human cell line. To process the ENCODE data, I converted all five files into pandas dataframes and filtered by the first column (condition was that `file[0] == 'chr21'`). Then, I converted all the "starts" and "ends" points of each factor's regional location (columns 1 and 2) into numpy arrays respectively. All the start arrays were put in the "starts" list and all the end arrays were put in the "ends" list. Then, I traversed across the chr21 FASTA text sequence in 1000 bp chunks using a while loop, filtering for 1000 bp chunks where the center 200 bps falls between the start and end of any of the five factors. For each index in starts & ends that overlapped with the 200 bps center, the corresponding index of a 5-index labels list was transformed into a 1. I additionally filtered for "unknown" nucleotides, denoted by an "N." If a 1000 bp region had more than 100 unknown nucleotides, it was automatically excluded. The remaining "N" nucleotides were one-hot-encoded as `<0, 0, 0, 0>`. Lastly, if the window passed all these filters it was appended to our "X" input, and the corresponding label was appended to our "Y" input. Then, the window slid over by 200 bps.

First Attempt

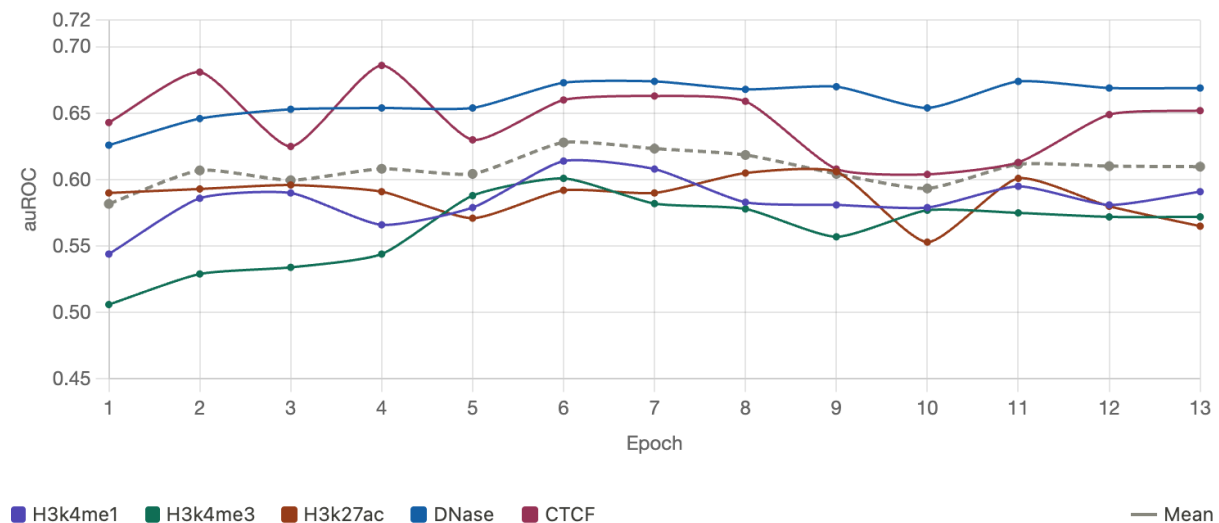
Instead of letting the full 30 epochs play out, I manually stopped the model after 13 epochs as there was an early plateau with a noisy validation signal, suggesting overfitting or the potential of overfitting. In other words, while the training loss was still decreasing slowly, the validation loss was just hovering and bouncing around 0.46. Essentially, it seemed that the model had stopped improving on held out data by the 7th epoch.

Speaking on the mark-specific auROC, which is essentially a measure of how well the model ranks positive windows over negative windows. An auROC close to 0.5 means the model is no better than random guessing. Our model's average auROC hovered around 0.6, with the DNase mark having the highest auROC hovering around 0.67. H3k4me3 and H3k27ac are broader, more diffuse histone modifications that correlate less directly with local, so weaker auROC for these is expected and consistent with what the original DeepSEA paper also found.

Train vs validation loss



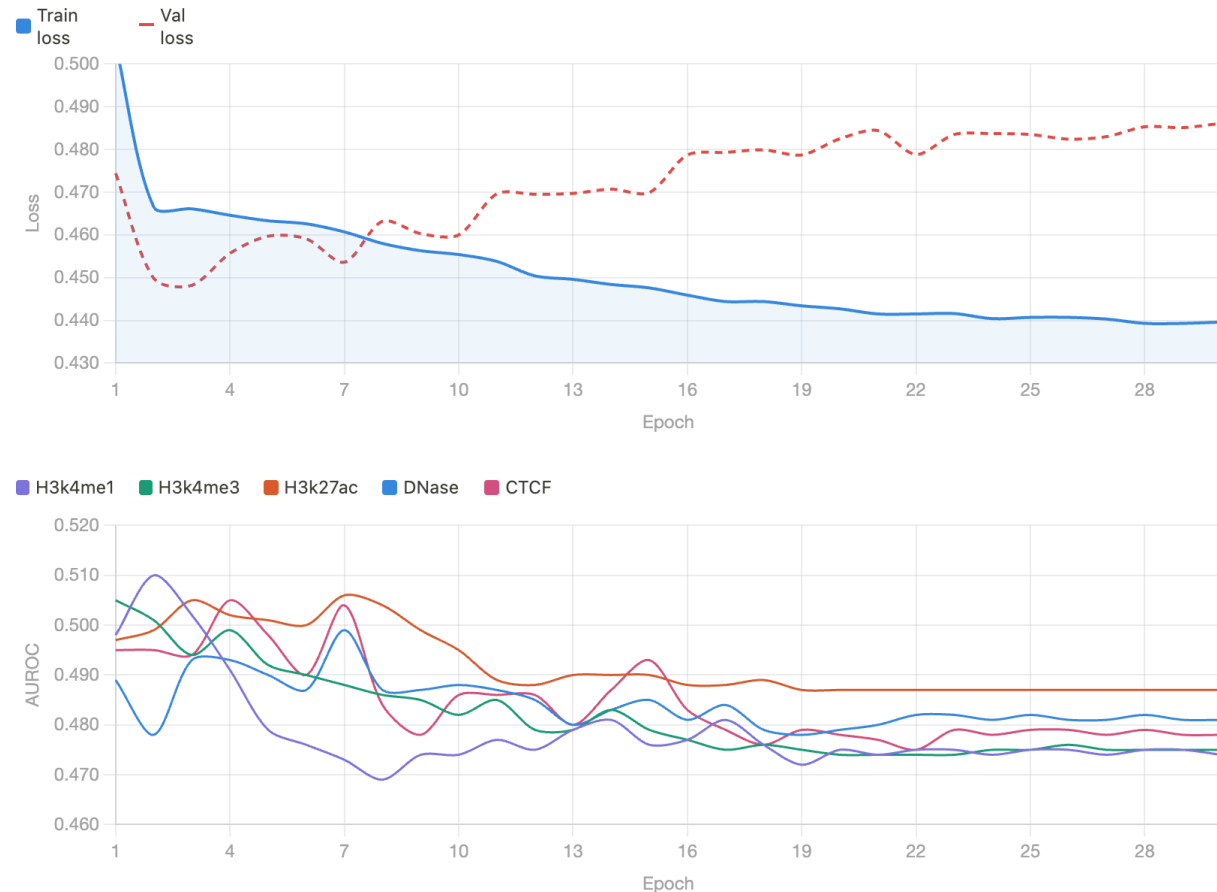
Validation auROC by mark



In terms of future improvement, I think the main issue with the first rendition of the model was the lack of chromosomal data. The original DeepSea, while tracking 919 features, does use all chromosomes but two for its training data, while my version only used ch21's data, the smallest chromosome. Additionally, I want more of a dynamic learning rate, which is essentially a measurement of how big the steps the optimizer takes are to adjust the weights. An overly large learning rate, as in my case, leads to overshooting, causing oscillation. Through my research, I came across "ReduceLROnPlateau," which tracks validation loss, and if it doesn't improve for a set number of epochs, it lowers the learning rate, allowing more precision.

Second & Third Attempt

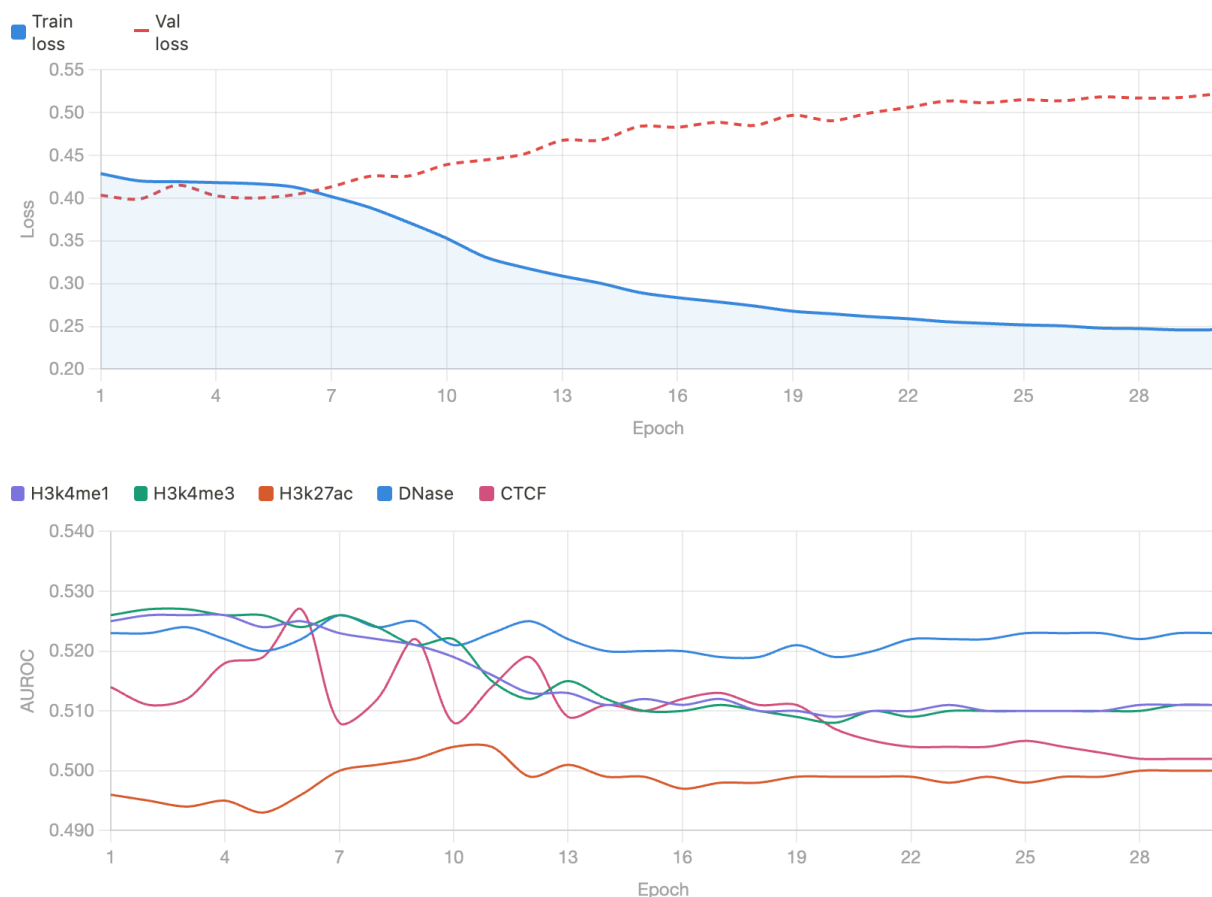
To recap, this time around, I added chr22 to the train/testing data to address the data limitation flagged after the first attempt, and ReduceLROnPlateau to have a more dynamic learning rate—essentially ensuring that the “steps” of weight adjustments get smaller and smaller to make the precise adjustments needed in later epochs.



Unfortunately, while the training loss decreased, the validation loss increased, which is, in other words, textbook overfitting. Simultaneously, the AUROC hovered around 0.48, which is essentially worse than random guessing. I accredited this poor performance to a model design choice where I had strayed from the original DeepSea. I used a the positional 80/20 split for training and testing, due to my scaled down data, and ultimately, that still leaked nearby windows across train/val. On the other hand, I chose to stray away from the original DeepSea architecture on another part. They focused on the set of 200-bp bins with at least one TF binding event, resulting in 521,636,200 bp of sequences (17% of whole genome), but I'm focusing on only 5 events/feature. Because these specific marks co-occur at the same genomic loci by biological design, a window that is positive for H3K27ac is very likely also positive for DNase and either H3K4me1 (enhancer) or H3K4me3 (promoter). This means within our positive windows, the label vectors are rarely sparse, the marks cluster together. With only 5 features, almost every positive window activates 2-3 marks simultaneously, leaving very few windows where the model can learn to distinguish, say, H3K27ac-positive from H3K27ac-negative while holding the other marks constant.

This is fundamentally different from DeepSEA's 919-feature setting, where even at an active enhancer locus, the window is still negative for hundreds of cell-type-specific or tissue-specific TF binding events. The sparsity of positive labels across 919 features gives the model rich discriminative signal within each window. With 5 biologically correlated marks, that within-window discrimination largely collapses, which explains why our per-mark AUROCs were so similar to each other and so hard to push above 0.5.

So, in turn, for the third attempt, I switched from a positional 80/20 split on chr21 to a proper chromosome-held-out split (train: chr19/21/22, test: chr18), in turn preventing leakage from nearby windows on the same chromosome across training and validation data. I also, deviating from DeepSea, added negative windows (1:1 ratio), hoping the model would better understand what the negative signals look like.



This time around, the training and validation loss differentiated much more significantly, again signaling over fitting and the AUROC plateaued at approximately 0.52 despite the model memorizing training data. One theory is that the chromosome splitting is a much more “real” test, in turn witnessing the worse performance.

The next logical step is more data. The original DeepSEA trained on essentially the entire human genome; our model has seen three chromosomes. With only chr19, chr21, and chr22 as training data, the model is being asked to generalize regulatory sequence grammar learned from a small, biased slice of the genome to an entirely unseen chromosome. Adding more chromosomes — or ideally training on the full genome minus the held-out test chromosomes — would be the single highest-impact improvement. Beyond data volume, adding L2 weight decay to the optimizer and increasing dropout would directly combat the overfitting observed in both runs. The co-occurrence problem with 5 correlated marks likely represents a ceiling that more data alone won't fully solve; expanding to a broader set of chromatin features, even a few dozen, would restore the within-window discriminative signal that DeepSEA relies on.